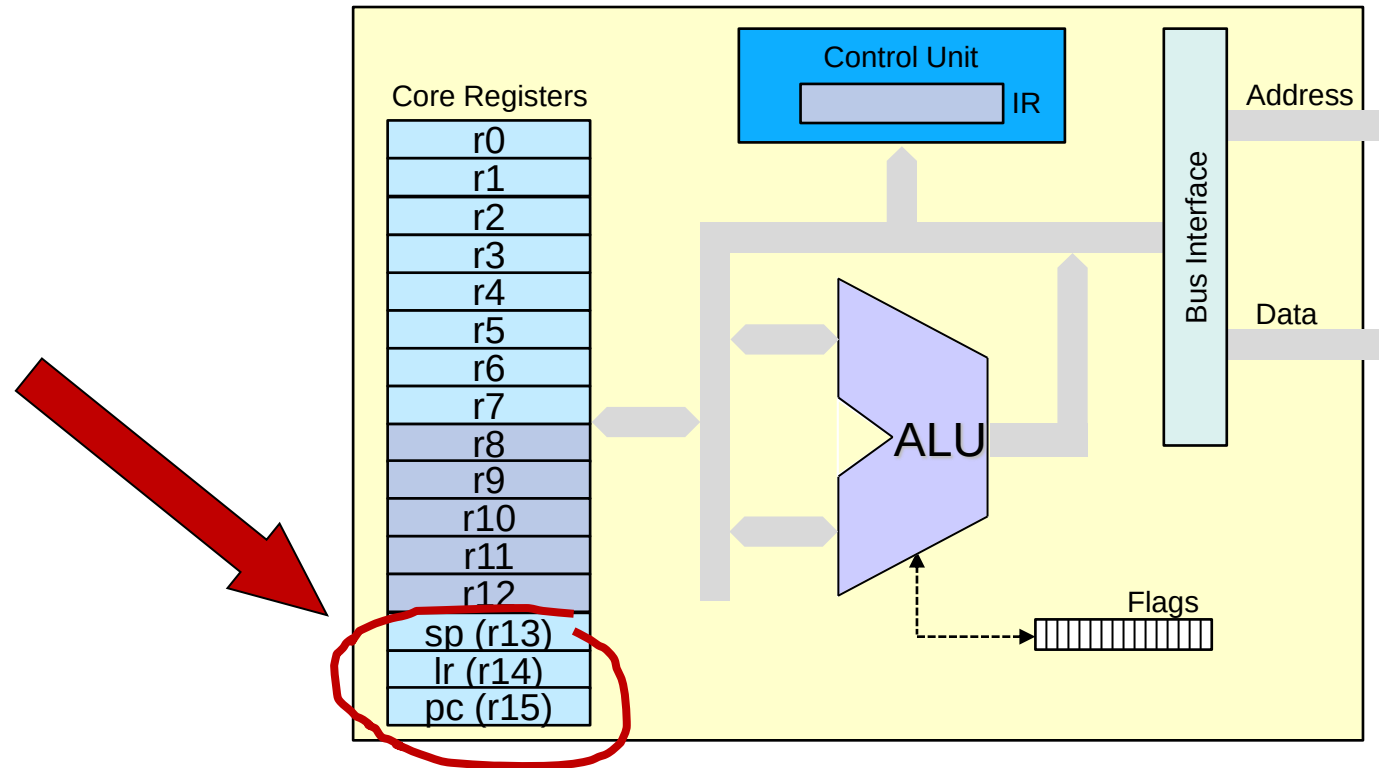


Subroutines and Stack

Computer Engineering 1

Motivation

- Do you remember?



Motivation

main program

```
...  
b   proc_a   ; 1. call  
...           ; next instruction 1  
...  
...  
...  
b   proc_a   ; 2. call  
...           ; next instruction 2  
...  
...  
...
```

subroutine

```
proc_a  
...  
...  
b   ???
```

```
graph LR  
    subgraph MainProgram [main program]  
        direction TB  
        M1["...  
b   proc_a   ; 1. call  
...           ; next instruction 1  
...  
...  
b   proc_a   ; 2. call  
...           ; next instruction 2  
...  
...  
..."]  
    end  
    subgraph Subroutine [subroutine]  
        direction TB  
        S1["proc_a  
...  
...  
b   ???"]  
    end  
    M1 -- "1. call" --> S1  
    S1 -- "return" --> M1  
    M1 -- "2. call" --> S1  
    S1 -- "return" --> M1
```

Agenda

- **Terminology**
- **Subroutine Call and Return**
- **Nested Subroutine Calls**
- **Stack**
- **ARM: push and pop**
- **Nested Subroutines (revisited)**
- **Instructions using sp**
- **Assembler Directives**

Learning Objectives

At the end of this lesson you will be able

- to explain and discuss the term subroutine
- to comprehend and explain how a subroutine call and return are implemented on ARM Cortex-M
- to implement (nested) subroutines in assembly
- to explain how a processor stack works
- to determine the content of the stack for a given assembly program with nested subroutine calls

Terminology

- Subroutine / Procedure / Function / Method
 - Sequence of instructions to solve a subtask
 - Called by "name"
 - Interface and functionality known
 - Internal design and implementation are hidden
 - information hiding
 - Can be called from miscellaneous places in the program
- Why Subroutines?
 - Basic element of structured programming
 - Reuse of the same implementation → less mistakes
 - Simplifies verification and maintenance
 - Requires less memory
 - only one instance for several calls

Terminology

Terms used by ARM

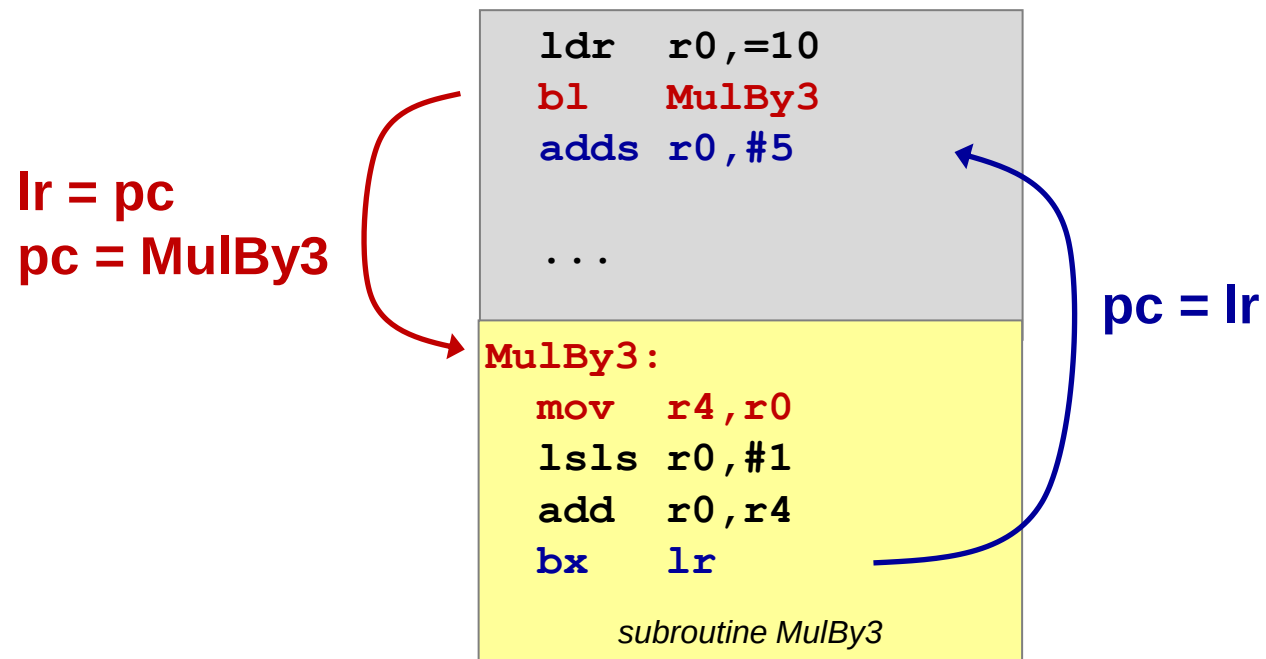
- Routine, subroutine
 - A fragment of program to which control can be transferred that, on completing its task, returns control to its caller at an instruction following the call. Routine is used for clarity where there are nested calls: a routine is the caller and a subroutine is the callee.
- Procedure
 - A routine that returns no result value.
- Function
 - A routine that returns a result value.

*source: Procedure Call Standard for the ARM Architecture
ARM IHI 0042E, 30th November 2012*

Subroutine Call and Return

Change of control flow

- Call Save pc to Link Register (lr)
- Return Restore pc from lr



Subroutine Call and Return

Structure of Subroutine

- Label with Name
 - e.g. **MulBy3**
- Return Statement
 - **bx lr**

MulBy3:

```
mov    r4,r0
lsls   r0,#1
add    r0,r4
bx    lr
```

Subroutine Call and Return Commands

- **bl <label>**
 - Store current pc in lr
 - Branch to <label>
 - $pc = pc +/- \text{offset}$
 - offset range -16'777'216 to 16'777'214

Subroutine call through a label

- unconditional
- **relative**
- **direct**

```
bl label T1  
label (imm23) ∈ { -8'388'608..8'388'607 } << 1  
  


|       |   |             |    |   |   |   |             |
|-------|---|-------------|----|---|---|---|-------------|
| 11110 | S | iiiiiiiiiii | 11 | J | 1 | K | iiiiiiiiiii |
|-------|---|-------------|----|---|---|---|-------------|

  
I1 = not(J xor S) I2 = not(K xor S)  
imm23 = s:i1:i2:imm10:imm11:0  
lr = pc | 1u;  
pc = pc + imm23 << 1;
```

Subroutine Call and Return

... Commands

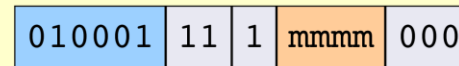
- **blx (register)**
 - Store current pc in lr
 - Address of subroutine in register
 - Branch
 - $pc = register$
 - Branch address from 0 to 2^{32}

Subroutine call with address in register

- unconditional
- **absolute**
- **indirect**

blx rm *T1*

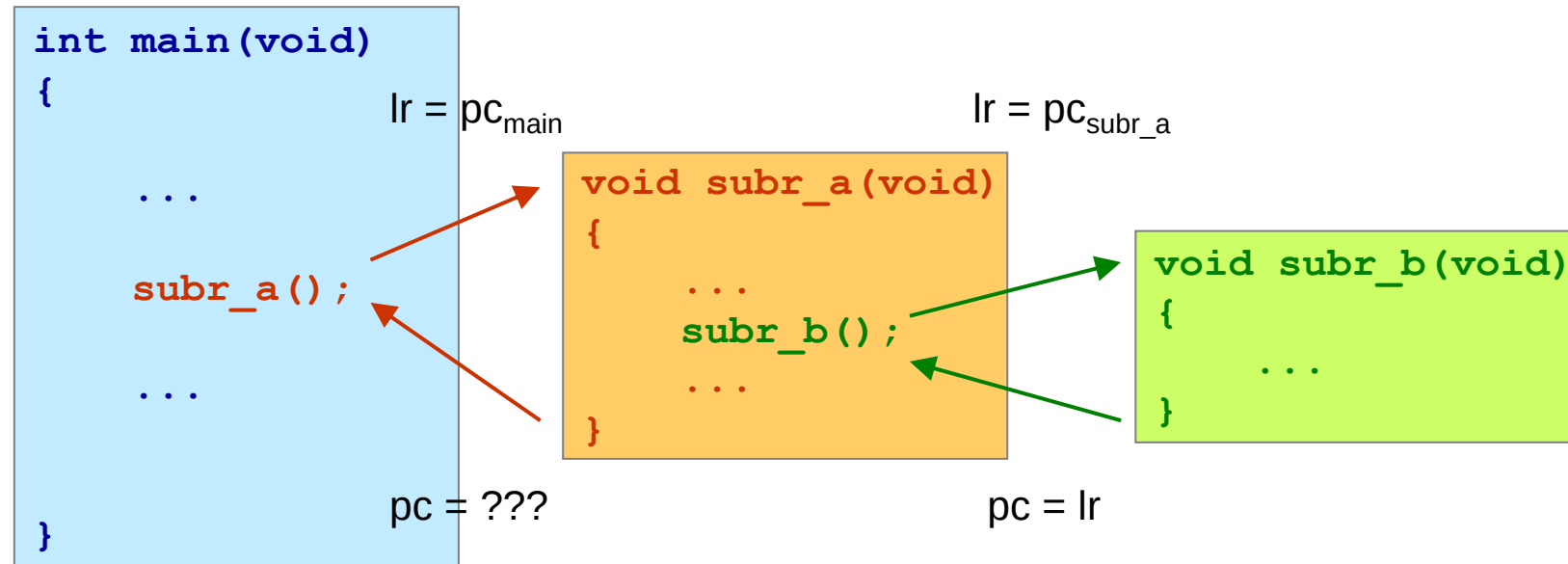
$rm \in \{ r0..r11 \}$



$lr = pc \mid lu;$
 $pc = rm \ \& \ \sim lu;$

Nested Subroutine Calls

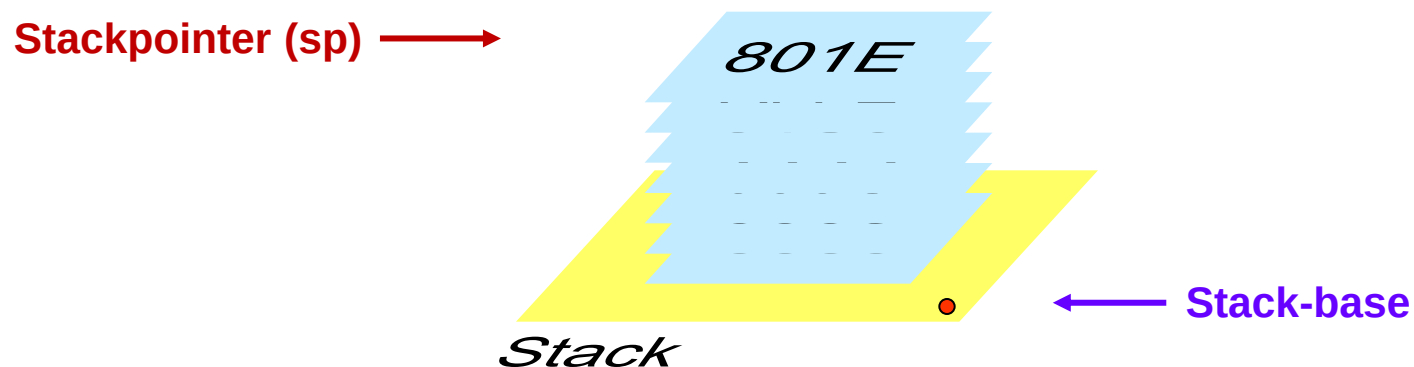
- Nested Subroutine (Function) Calls
 - How do we do that with a single lr?



Stack

Stack as Object

- Methods
 - push() and pop()
- Data
 - pushed (written) on top of the stack
 - popped (fetched, read) from the top of the stack → LIFO¹⁾



¹⁾ Last In First Out

Stack Implementation

- **Stack Area (Section)**
- **Stack Pointer (sp)**
- **push { ... }**
- **pop { ... }**
- Direction on ARM
- Alignment

Continuous area of RAM

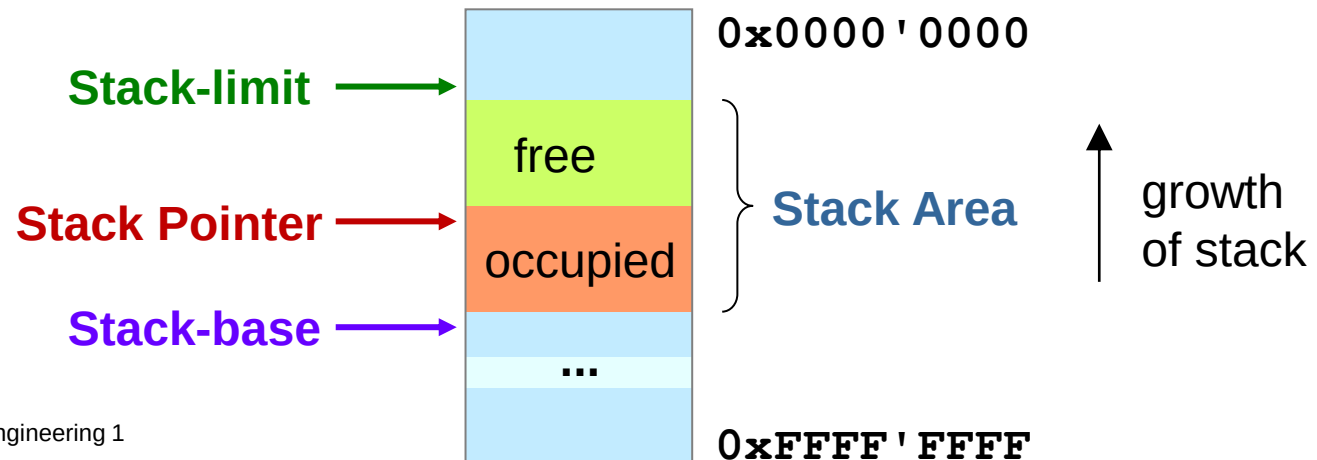
r13 → points to last written data value

Decrement sp and store word(s)

Read word(s) and increment sp

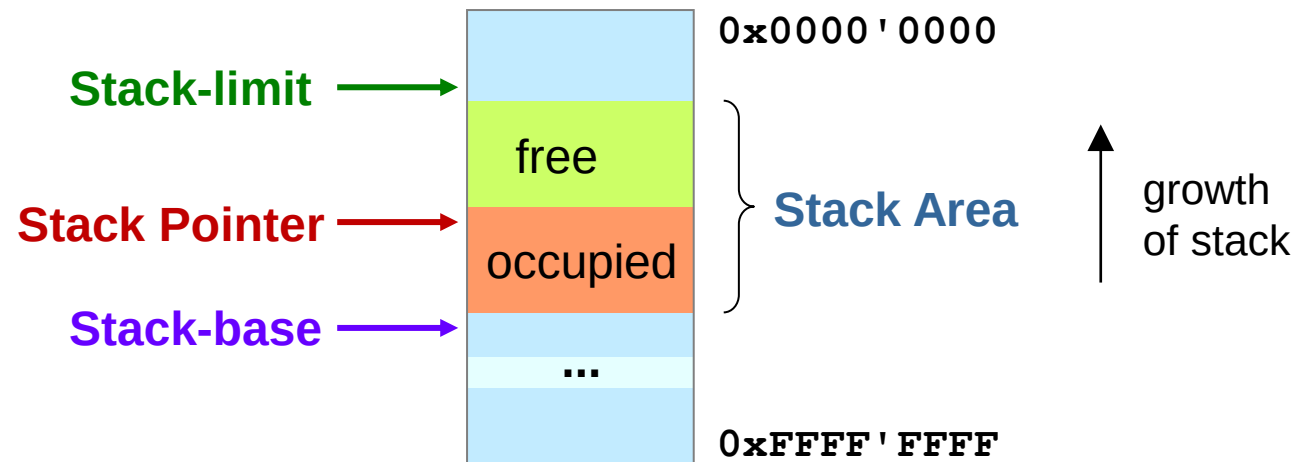
"grows" from higher towards lower addresses
→ full-descending stack

Stack operations are **word-aligned**



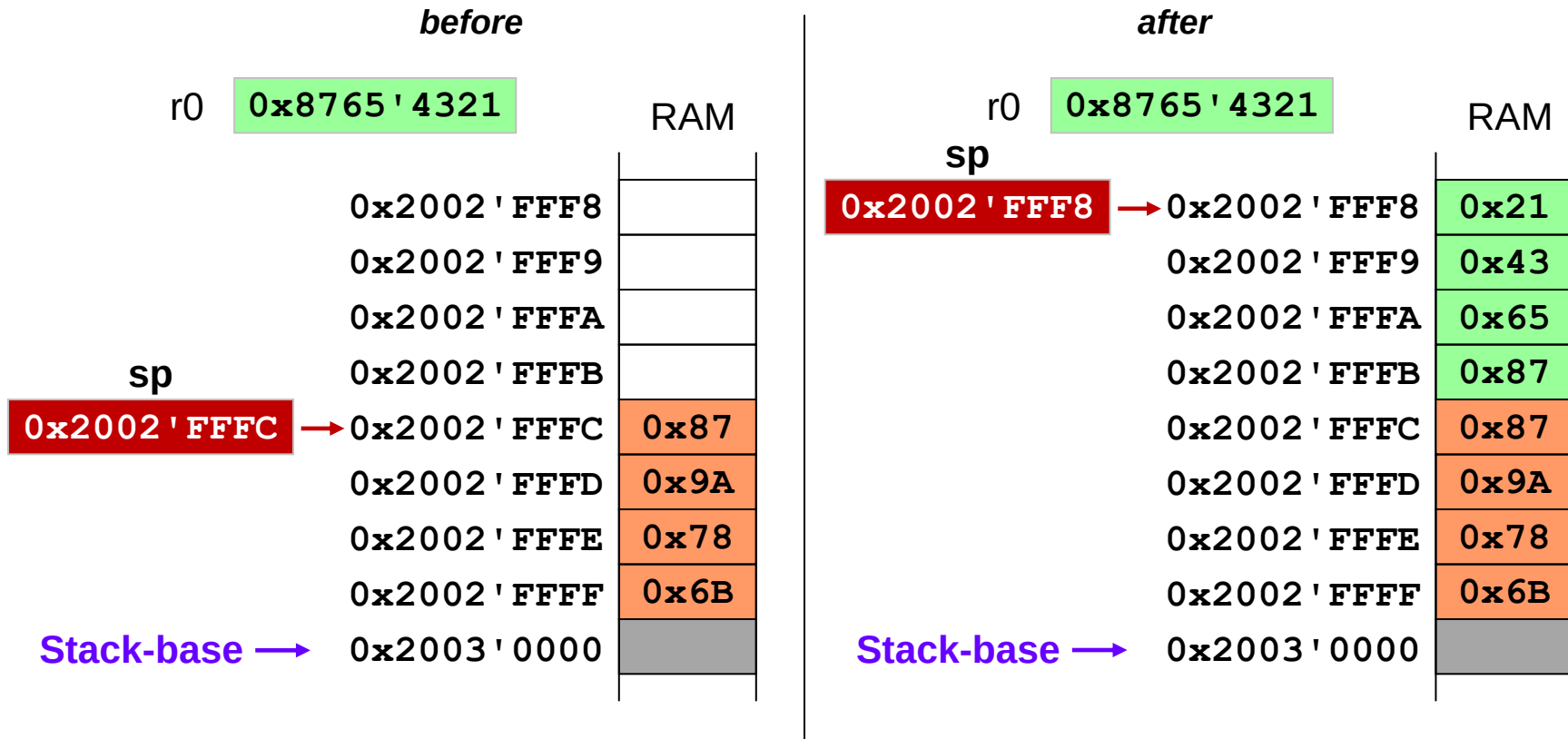
Stack Initialization

- Processor fetches initial value of sp (**Stack-base**) at reset
 - from address 0x0000'0000
- **Stack-base** is right above the stack area
 - sp is decremented before writing the first word



ARM: push and pop

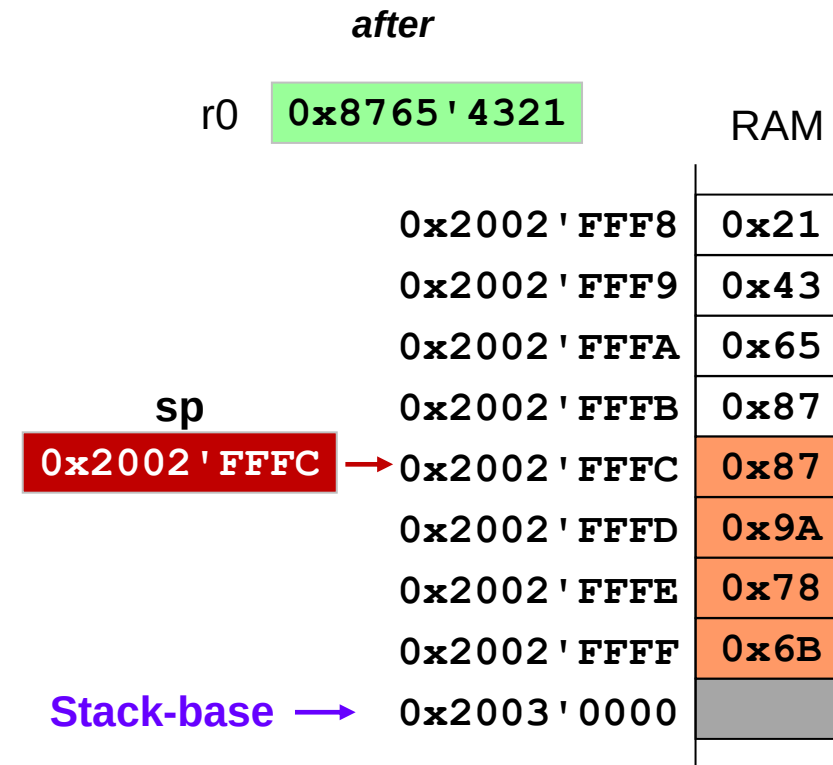
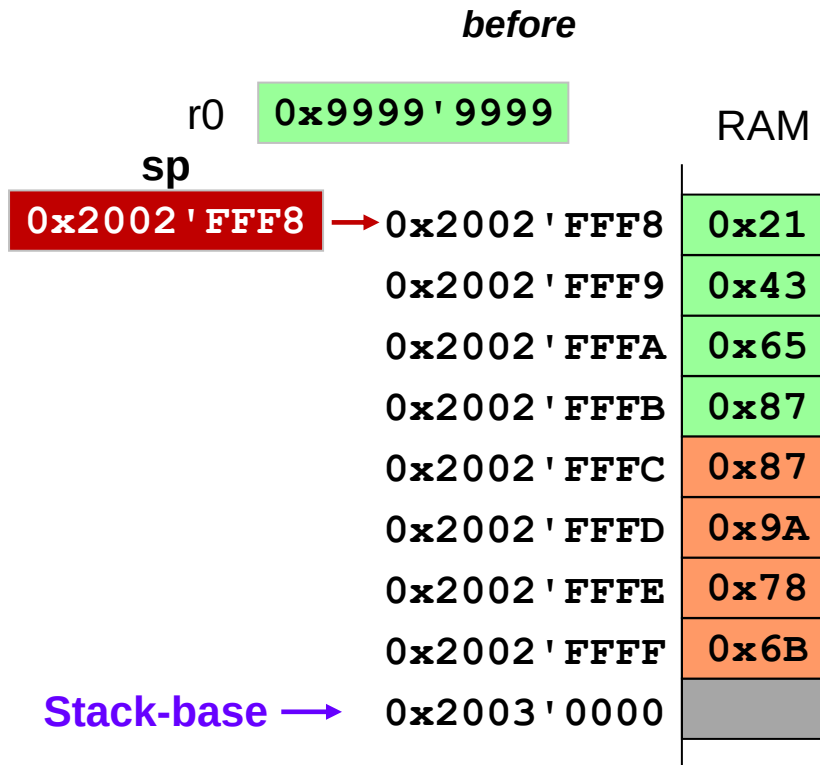
Example: **push {r0}**



sp points to last value that has been written

ARM: push and pop

Example: **pop {r0}**



ARM: push and pop

push

- **registers**
 - One or more registers to be stored
 - Low registers
 - **reg_list** = one bit per register
 - lr (r14) → M-bit
 - No other high registers
 - Lowest register stored first (lowest address)

```
push { registers }  
registers  $\subseteq$  { r0..r7, lr }
```

T1

1011	0	10	1	rrrrrrrr
------	---	----	---	----------

```
for (k = 0; k < #regs; k++){  
    stack[k] = regs[k];  
}
```

00000000	B480	push	{ r7 }
00000002	B43A	push	{ r1, r3, r4, r5 }
00000004	B43A	push	{ r1, r3-r5 }
00000006	B500	push	{ lr }
00000008	B580	push	{ r7, lr }

M::reg_list = 0x03A
= 0'0011'1010b

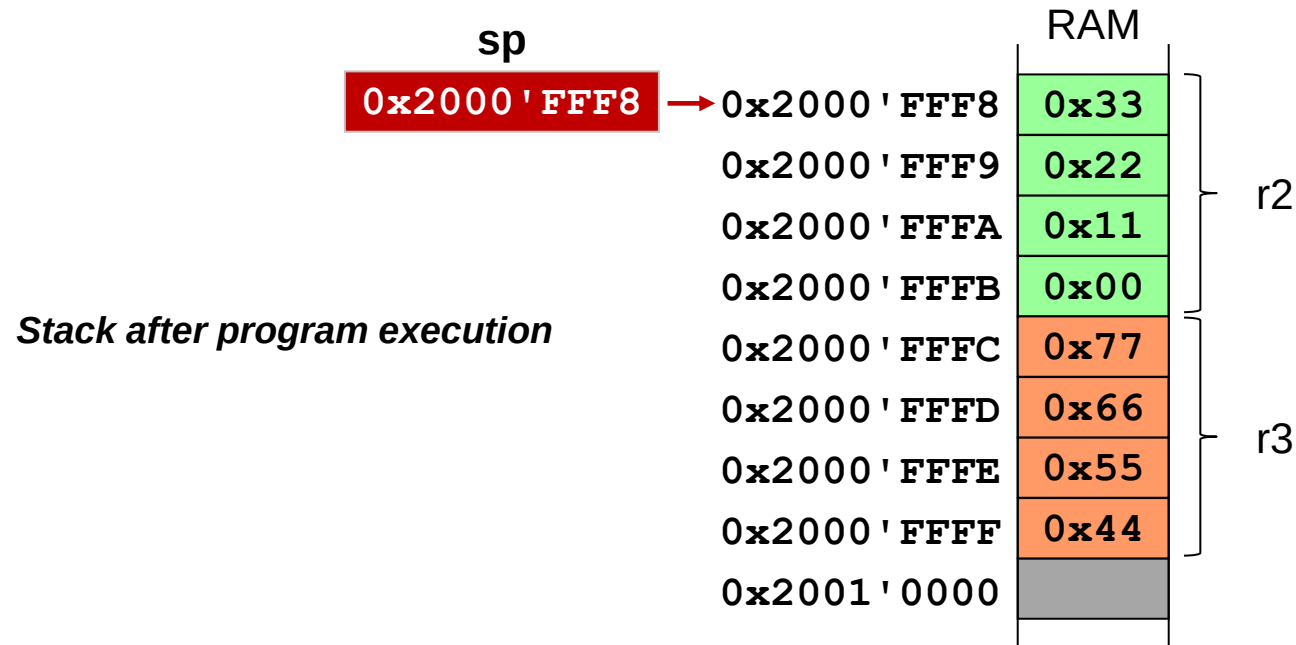
ARM: push and pop

Storage Order push

- Lowest register
 - stored to lowest address ¹⁾

Example

```
ldr    r1,=0x20010000
mov    sp,r1
ldr    r2,=0x00112233
ldr    r3,=0x44556677
push   {r2,r3}
```



¹⁾ The lowest register is stored first.

ARM: push and pop

- **registers**
 - One or more registers to be restored
 - Low registers
 - **reg_list** = one bit per register
 - pc (r15) → P-bit
 - No other high registers
 - Lowest register reloaded first

```
pop { registers } T1  
registers  $\subseteq$  { r0..r7, pc }
```

1011	1	10	p	rrrrrrrr
------	---	----	---	----------

```
for (k = 0; k < #regs; k++){  
  regs[k] = stack[k];  
}
```

00000000	BC80	pop	{r7}
00000002	BC3A	pop	{r1, r3, r4, r5}
00000004	BC3A	pop	{r1, r3-r5}
00000006	BD00	pop	{pc}
00000008	BD80	pop	{r7, pc}

P::reg_list = 0x03A
= 0'0011'1010b

ARM: push and pop

- **ARM Stack**
 - Only Words $\rightarrow$ 32-bit
 - Pushing and popping of half-words and bytes not possible
 - I.e. $sp \bmod 4 = 0 \rightarrow$ word aligned
- **"Number of pushes" = "Number of pops"**
- **Stack-limit** < **sp** < **stack-base**
 - Stack size has to fit program requirements

Nested Subroutines (revisited)

Save lr on Stack

```
addR_LED_31_0 = 0x60000100  
LED_PATTERN = 0xA55A5AA5
```

subrExample:

```
push    {r4,r5,lr}
```

Save lr and registers used
by subroutine

@ write pattern to LEDs

```
ldr     r4,=addR_LED_31_0
```

```
ldr     r5,=LED_PATTERN
```

```
str     r5,[r4]
```

Call another subroutine

```
bl      write7seg
```

```
pop     {r4,r5,pc}
```

Restore registers and pc

Please note: **bx lr** is not required here, as we directly restore the pc using **pop**

Instructions using sp

- **Add to / subtract from sp**
 - Immediate offset <imm>
 - Offset range 0 – 1020d and 0 – 508d respectively

load stack pointer
plus offset into
register

add (sp plus immediate)

```
add rd, sp, #imm8
```

T1

rd ∈ { r0..r7 }
imm8 ∈ { 0..255 } << 2

1010

1

ddd

iiiiiii

rd = sp + imm8 << 2;

allocate (sub) /
deallocate (add)
memory on stack

```
add sp, #imm7
```

T2

imm7 ∈ { 0..127 } << 2

1011

0000

0

iiiiiii

sp = sp + imm7 << 2;

sub (sp minus immediate)

```
sub sp, #imm7
```

T1

imm7 ∈ { 0..127 } << 2

1011

0000

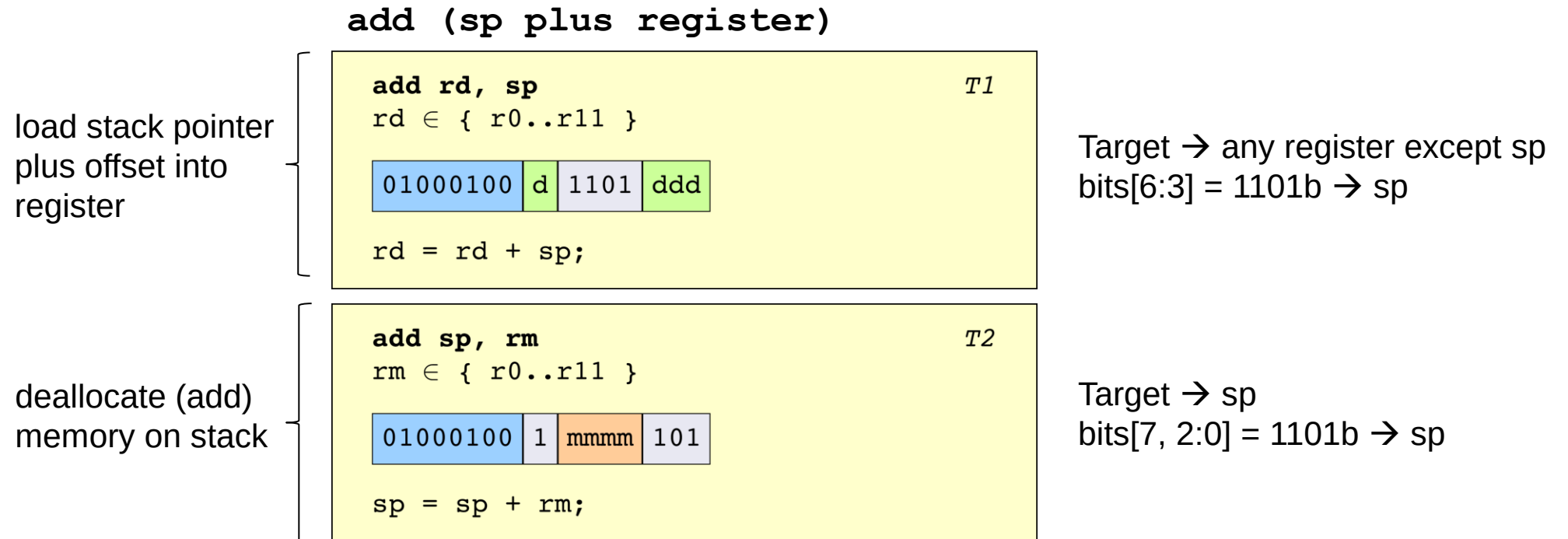
1

iiiiiii

sp = sp - imm7 << 2;

Instructions using sp

– Add to sp (Register)



Instructions using sp

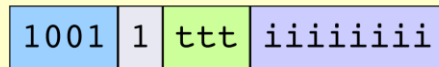
- Instructions with Opcodes previously covered

```
cmp  sp, Rm  
cmp  Rn, sp  
mov  sp, Rm  
mov  Rd, sp
```

Instructions using sp

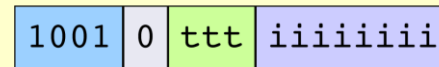
- **Accessing Memory using sp**
 - Immediate offset <imm>
 - Offset range 0 – 1020d
 - Word transfers

```
ldr rt, [ sp, <#imm8> ]           T2
rt ∈ { r0..r7 }
imm8 ∈ { 0..255 } << 2
```



```
rt = *((uint32_t*)(sp + imm8 << 2));
```

```
str rt, [ sp, <#imm8> ]           T2
rt ∈ { r0..r7 }
imm8 ∈ { 0..255 } << 2
```



```
((uint32_t*)(sp + imm8 << 2)) = rt;
```

Instructions using sp

- Using other instructions to implement ¹⁾

- push {r2,r3,r6}

```
sub    sp, sp, #12
str    r2, [sp]
str    r3, [sp, #4]
str    r6, [sp, #8]
```

- pop {r2,r3,r6}

```
ldr    r2, [sp]
ldr    r3, [sp, #4]
ldr    r6, [sp, #8]
add    sp, sp, #12
```

¹⁾ Caveat: the alternate implementations are not atomic

Assembler Directives for Functions

- **Assembler Directives**
 - `.type name, %function`
 - `.size name, . - name`
- **Mark start and end of a procedure / function**
 - Used by debugger (tool)
 - Buttons "step over" and "step out"
 - Structure code for reader

```
myfunc:  
    push    { ...,lr}  
    ...  
    ...  
    pop     { ...,pc}
```

Conclusions

- **Subroutines**
 - Structured programming
 - Avoids duplicated code / clear interface
 - Call and return on ARM:
 - `bl <label>` and `bx lr / pop {pc}`
 - Nested subroutines → save `lr` on stack
- **Stack**
 - Continuous area of memory → Last-in First-Out
 - `push` und `pop`
 - ARM
 - Full-descending stack
 - `sp` points to last entry that has been written
 - grows from higher towards lower addresses